

j4_controller

Firmware for the **Johnny 4 robot controller/transmitter**. Runs on a LILYGO TTGO T-Display v1.1 (ESP32). Reads potentiometers and two 4x4 keypads, ingests four more pots from the j4_display_right board, transmits control data to the robot receiver over ESP-NOW, and mirrors live data to the two XIAO display boards over UART.

What is Johnny 4?

Johnny 4 is a prop robot controlled wirelessly. This board is the handheld controller that the operator uses to drive the robot's servos, select audio phrases, and monitor battery status.

What this firmware does

- Reads analog potentiometers via four local ADS1115 ADC modules: the eyes joystick (eyes_x / eyes_y), the neck joystick (neck X / jaw Y), and the eight middle face pots (Eyebrow L/R, Basket Eyebrow L/R, Nose, Nose Basket, Bottom Eyelid L/R); the fourth module carries the neck-pivot pot and the two linear faders
- Reads the four panel toggle switches (LASER, VENT, EYE POP, and the AUX toggle by the right joystick) on GPIOs 32/33/13/15 with internal pull-ups
- Ingests four more pots (iris, color, brightness, volume) from [j4_display_right](#)'s dedicated ADS1115 over Serial2 as 25 Hz P: lines
- Reads two 4x4 matrix keypads via PCF8574 I2C expanders using a custom scan routine
- Left keypad: jukebox-style audio phrase selection: press a letter (A-D) then a digit (0-9) to queue a phrase, or press a digit alone to queue a 0x phrase (e.g. pressing 8 queues "08.wav")
- Right keypad: face presets. Tap a key to recall its saved face; hold a key 3 seconds to save the current face to it (j4_display_right shows a confirm prompt, * confirms). Faces persist on j4_talk's microSD and are re-loaded in the background whenever the talk link is up
- Press * to send a STOP command, press # to clear the buffer
- Mixes the joystick into differential neck-left / neck-right values over a 0-3200 range
- Transmits all control data to the robot receiver via ESP-NOW (fixed-size packed structs, no String members so they survive the wireless memcpy)
- Receives the jukebox file list from the robot receiver in chunks and forwards it to the display board; carries a need_filelist flag so the receiver re-sends the list if this board boots late
- Answers LIST? requests from the display board out of its cached copy, so a display reboot recovers the list without a full round trip
- Displays live pot values, keypress state, and battery voltage on the built-in TFT
- The TTGO's built-in button (GPIO 35) cycles the TFT through three screens: live data, WiFi MAC address, and a connection-status screen showing ESP-NOW LINK, j4_stepper_neck, j4_stepper_eyes, j4_talk, j4_display_left, and j4_display_right as CONNECTED (green) / DISCONNECTED (red)
- Sends a 49-byte binary status packet to the j4_display_left board at 25 fps over UART
- Shows a STATUS line on the built-in TFT: "ONLINE" when the ESP-NOW link is up and the steppers are healthy, "OFFLINE" if no status packet arrives, or the reported stepper fault (e.g. "NL OT", "EYES OFFLINE"); green when healthy, red on any fault

Hardware

Component	Details
Microcontroller	LILYGO TTGO T-Display v1.1 (ESP32 with built-in 1.14" ST7789 TFT)
ADC modules	Four ADS1115 (I2C addresses 0x48, 0x49, 0x4A, 0x4B); a fifth lives on j4_display_right
Toggle switches	LASER, VENT, EYE POP, AUX -- GPIOs 32/33/13/15, INPUT_PULLUP, switch closes to GND

Keypad expanders	Two PCF8574 I2C I/O expanders: 0x21 (keypad_left, A0 jumper high) and 0x20 (keypad_right)
Keypads	Two 4x4 matrix keypads, headers soldered directly into P0-P7; left = phrase select, right = face presets
Displays	j4_display_left (jukebox, Serial1) + j4_display_right (pot labels, Serial2)

Pin assignments

GPIO	Function
21	I2C SDA (ADS1115 x4, PCF8574 keypads x2)
22	I2C SCL
17	Serial1 TX to j4_display_left (XIAO D7)
27	Serial1 RX from j4_display_left (XIAO D6)
25	Serial2 TX to j4_display_right (XIAO D7, face-preset messages)
26	Serial2 RX from j4_display_right (XIAO D6, pot feed)
32	LASER toggle (INPUT_PULLUP, switch closes to GND)
33	VENT toggle (INPUT_PULLUP, switch closes to GND)
13	EYE POP toggle (INPUT_PULLUP, switch closes to GND)
15	AUX toggle, next to the right joystick (INPUT_PULLUP, unassigned)
34	Battery voltage sense (ADC input, read-only)
35	Screen-cycle button (TTGO built-in; data / MAC / connection status)

GPIO 35 is the TTGO's on-board button, so it needs no wiring. Potentiometer inputs are routed through the ADS1115 modules. GPIO 12 causes boot/flash issues when connected to a pot and is avoided. GPIO 17 is unreliable as an input and is used only as TX. Pins 36, 37, and 38 are input-only. The four toggles use the internal pull-ups, so each switch just shorts its GPIO to GND when flipped ON -- no external resistors.

Pin diagram

Physical layout, display facing AWAY from you (rails visible), USB-C at the TOP (matching how the board is mounted). The two header rails read top-to-bottom as shown.

LEFT rail		RIGHT rail
	+-- [USB-C] --+	
I2C SDA (ADS x4 + keypad) -- 21		5V
I2C SCL -- 22		GND ground
DISP-L TX -> XIAO D7 -- 17	TTGO	27 DISP-L RX <- XIAO D6
(free) -- 2	T-Display	26 DISP-R RX <- XIAO D6
AUX toggle -- 15	v1.1	25 DISP-R TX (face msgs)
EYE POP toggle -- 13		33 VENT toggle
(avoid: boot/flash) -- 12	[ST7789	32 LASER toggle

```

ground -- GND|   TFT on   | 39  (input only)
3.3V  -- 3V3|   back ]   | 38  (input only)
5V    -- 5V |           | 37  (input only)
ground -- GND|           | 36  (input only)
+-----+

```

on-board (no header pin): GPIO34 = battery sense, B00T = GPIO0,
 GPIO35 = screen-cycle button, TFT on 4/5/16/18/19/23
 I2C bus: ADS_01 @ 0x48, ADS_02 @ 0x49, ADS_03 @ 0x4A, ADS_04 @ 0x4B,
 PCF8574 keypad_left @ 0x21, keypad_right @ 0x20
 toggles: INPUT_PULLUP, switch closes to GND (ON = LOW)

ESP-NOW (wireless) links this board to j4_receiver.

ADS1115 pin diagrams

All four modules share the TTGO's I2C bus (SDA = GPIO 21, SCL = GPIO 22, 3.3V, GND); only the ADDR strap and the four analog channels differ. Every pot's wiper goes to its A-pin; the outer legs go to 3.3V and GND.

ADS_01 @ 0x48 (ADDR -> GND) four face pots, left bank

```

+-----+
| VDD  | 3.3V
| GND  | GND
| SCL  | TTGO GPIO 22 (I2C SCL)
| SDA  | TTGO GPIO 21 (I2C SDA)
| ADDR | GND = address 0x48
| ALRT | not connected
| A0   | Eyebrow L pot wiper      -> PCA9685 ch 6
| A1   | Eyebrow R pot wiper      -> PCA9685 ch 7
| A2   | Basket Eyebrow L pot wiper -> PCA9685 ch 8
| A3   | Basket Eyebrow R pot wiper -> PCA9685 ch 9
+-----+

```

ADS_02 @ 0x49 (ADDR -> VDD) both joysticks

```

+-----+
| VDD  | 3.3V
| GND  | GND
| SCL  | TTGO GPIO 22 (I2C SCL)
| SDA  | TTGO GPIO 21 (I2C SDA)
| ADDR | VDD = address 0x49
| ALRT | not connected
| A0   | eyes joystick X wiper    -> eyes pan servo (PCA9685 ch 3)
| A1   | eyes joystick Y wiper    -> eyes tilt servo (PCA9685 ch 4)
| A2   | neck joystick X wiper     -> neck L/R differential mix
| A3   | neck joystick Y (jaw) wiper -> neck L/R base height
+-----+

```

ADS_03 @ 0x4A (ADDR -> SDA) four face pots, right bank

```

+-----+
| VDD  | 3.3V
| GND  | GND
| SCL  | TTGO GPIO 22 (I2C SCL)
| SDA  | TTGO GPIO 21 (I2C SDA)

```

```

| ADDR | SDA = address 0x4A
| ALRT | not connected
| A0   | Nose pot wiper (up/down)      -> PCA9685 ch 10
| A1   | Nose Basket pot wiper         -> PCA9685 ch 11
| A2   | Bottom Eyelid L pot wiper     -> PCA9685 ch 12
| A3   | Bottom Eyelid R pot wiper     -> PCA9685 ch 13
+-----+

ADS_04 @ 0x4B (ADDR -> SCL)          neck-pivot + linear faders
+-----+
| VDD  | 3.3V
| GND  | GND
| SCL  | TTGO GPIO 22 (I2C SCL)
| SDA  | TTGO GPIO 21 (I2C SDA)
| ADDR | SCL = address 0x4B
| ALRT | not connected
| A0   | neck-pivot pot wiper          -> nP on the stepper link (0-3200)
| A1   | fader_left wiper              -> Nose Basket (doubles ADS_03 A1)
| A2   | fader_right wiper             -> iris (doubles j4_display_right IRIS)
| A3   | spare
+-----+

```

Analog inputs

Local, on the four ADS1115 modules over I2C:

Module	Channel	Control
ADS_01 (0x48)	A0	Eyebrow L pot
ADS_01 (0x48)	A1	Eyebrow R pot
ADS_01 (0x48)	A2	Basket Eyebrow L pot (was eye-pop, now a toggle)
ADS_01 (0x48)	A3	Basket Eyebrow R pot
ADS_02 (0x49)	A0	Eyes joystick X / eyes_x (was left arm)
ADS_02 (0x49)	A1	Eyes joystick Y / eyes_y (was right arm)
ADS_02 (0x49)	A2	Neck joystick X
ADS_02 (0x49)	A3	Neck joystick Y (jaw, feeds neck mixer)
ADS_03 (0x4A)	A0	Nose pot (up/down)
ADS_03 (0x4A)	A1	Nose Basket pot (up/down)
ADS_03 (0x4A)	A2	Bottom Eyelid L pot
ADS_03 (0x4A)	A3	Bottom Eyelid R pot
ADS_04 (0x4B)	A0	Neck-pivot pot (silver knob below j4_display_left, 0-3200)
ADS_04 (0x4B)	A1	fader_left (linear fader, doubles the Nose Basket pot)

ADS_04 (0x4B)	A2	fader_right (linear fader, doubles the IRIS pot)
ADS_04 (0x4B)	A3	Spare

The eight face pots ride the ESP-NOW control packet to PCA9685 channels 6-13 on j4_receiver (Eyebrow L/R = ch 6/7, Basket Eyebrow L/R = ch 8/9, Nose = ch 10, Nose Basket = ch 11, Bottom Eyelid L/R = ch 12/13). The neck-pivot pot rides the packet in its own field and leaves the receiver as the `nP` slot of the stepper stream.

The two linear faders each double an existing rotary pot, arbitrated last-mover-wins: whichever control of the pair moved last (beyond a small claim threshold) is the active source and its value is used, so the two never fight. `fader_right` pairs with the IRIS pot on `j4_display_right`; `fader_left` pairs with the Nose Basket pot. A source that is absent (module unplugged, display feed down) can neither claim nor hold active status.

Remote, streamed from `j4_display_right`'s dedicated ADS1115 (raw counts over Serial2, scaled here with the same `processPot()`):

Channel	Control	Destination
A0	Iris	270-deg iris servo (PCA9685 on j4_receiver)
A1	Color	WS2812B strip color (j4_receiver)
A2	Brightness	WS2812B strip brightness (j4_receiver)
A3	Volume	j4_talk audio volume

`eyes_x`, `eyes_y`, and `iris` drive servos on the receiver's PCA9685. The neck joystick mixes into neck-L / neck-R as before. `color` and `brightness` drive the receiver's WS2812B strip.

Toggle switches

Four panel toggles, each wired between its GPIO and GND (internal pull-up, ON = LOW). Their states travel in the control packet as a bitmask:

GPIO	Toggle	Bit	Action on the robot
32	LASER	0	Laser servo, PCA9685 ch 14 on j4_receiver
33	VENT	1	Vent-fin servo, PCA9685 ch 15 on j4_receiver
13	EYE POP	2	Eye-pop steppers: sends 0 (normal) or 3200 (popped) down the stepper chain (j4_stepper_neck to j4_stepper_eyes)
15	AUX	3	Next to the right joystick; read and transmitted, not yet assigned

Keypad wiring

Two 4x4 keypads, each on its own PCF8574 backpack on the shared I2C bus:

- **keypad_left** (the newer keypad, left of the panel) at **0x21** -- set the backpack's A0 jumper high. If that backpack turns out to be a PCF8574A, the same jumper lands at 0x39; change the two `0x21` s in `main.cpp`.
- **keypad_right** (the original keypad) at **0x20** -- all jumpers low.

The left keypad drives phrase select; the right keypad drives face presets (tap = recall, 3s hold = save prompt, * confirms). Each can be absent or hot-plugged without affecting the other. Each has its own keymap string

(`keymap_left` starts as a copy of `keymap_right` ; if the new model's matrix ordering differs, re-derive just that string by pressing each key and rearranging).

Each keypad is plugged straight into its PCF8574 P0-P7 (keypad pin 1 into P0, in order). The I2CKeyPad library's fixed scan pattern does not match this wiring, so the firmware uses a custom scan via the PCF8574 library directly.

Actual pin-to-wire mapping (derived on the original keypad):

PCF pin	Keypad wire
P0	Column 3 (3, 6, 9, #)
P1	Column 2 (2, 5, 8, 0)
P2	Column 1 (1, 4, 7, *)
P3	Row 4 (*, 0, #, D)
P4	Row 3 (7, 8, 9, C)
P5	Row 2 (4, 5, 6, B)
P6	Row 1 (1, 2, 3, A)
P7	Column 4 (A, B, C, D)

The scanner drives P0, P1, P2, P7 one at a time LOW and reads P3, P4, P5, P6.

Face presets

A **face** is a snapshot of the robot's facial expression: iris, eye color, eye brightness, the eight middle face pots, and the LASER / VENT / EYE POP toggle states. Volume, the neck values, and eyes X/Y are deliberately not part of a face.

On the **right keypad**:

- **Tap a key**: recall that key's saved face. Every face channel jumps to the saved value and holds there; turning any face pot takes just that channel back (frozen-baseline takeover, so even a slow turn works), and flipping a toggle takes that toggle back. The rest of the face stays put.
- **Hold a key for 3 seconds** (any key except `*`): `j4_display_right` shows "SAVE FACE ON `<key>` ? PRESS * TO CONFIRM". `*` saves, any other key cancels, and an unanswered prompt times out after 10 seconds.

Faces persist in `FACES.TXT` on `j4_talk`'s microSD, so they survive power-off. The sync is fully background and never blocks anything:

- After boot (or whenever the talk link appears) this board re-requests the saved-face dump every 2.5s until a complete one lands.
- A save made while `j4_talk` is offline stays in RAM flagged dirty ("PENDING SD" on the display) and is pushed automatically when the link comes up; the display shows "FACE `<key>` ON SD" when the Teensy confirms the write.
- If the Teensy reports an SD write failure, the face stays usable in RAM for the session and auto-retry stops until it is re-saved.

Slots are keyed by the keypad **character**, so re-deriving a keymap keeps every saved face on the same printed key. 15 keys are usable as slots (`*` is the confirm key). Face traffic is its own ESP-NOW packet type (0x04) translated by `j4_receiver` to text lines on the Teensy UART; see those repos for the protocol.

Dependencies

Managed via PlatformIO (`platformio.ini`):

Library	Version
RobTillaart/PCF8574	0.3.9
RobTillaart/ADS1X15	0.3.13
TFT_eSPI	bundled in <code>lib/</code> (pre-configured for TTGO T-Display v1.1)

Built-in ESP-IDF components used: `esp_now` , `WiFi` .

Building and uploading

```
# Build
pio run

# Upload
pio run --target upload

# Monitor serial output
pio device monitor
```

Upload speed is 921600 baud. Serial monitor is 115200 baud. Both are set in `platformio.ini` .

The TFT_eSPI copy in `lib/TFT_eSPI/` is pre-configured for the TTGO T-Display v1.1. Do not replace it with the registry version without reconfiguring.

Receiver

The robot receiver's MAC address is set in `broadcastAddress[]` near the top of `main.cpp` . Update this if the receiver board is swapped.

Related projects

- [j4_display_left](#) -- the XIAO ESP32S3 jukebox display (portrait, left of the panel)
- [j4_display_right](#) -- the XIAO ESP32S3 pot-label display (landscape, right of the panel) that streams the iris/color/brightness/volume pots to this board
- [j4_receiver](#) -- the robot-side TTGO that receives this controller's ESP-NOW packets and returns status